

Assignment 2-INFS343

Part I: Simulations

1. Simulate 10 standard Normal random numbers and screenshot your code and the results
Please note that the code provided was not complete. How to check the result? What was missing?

```
> set.seed(123)
> x <- rnorm(10, mean = 0, sd = 1)
> x
[1] -0.56047565 -0.23017749  1.55870831  0.07050839  0.12928774  1.71506499  0.46091621 -1.26506123
[9] -0.68685285 -0.44566197
>
```

2. Setting the random number seed with `set.seed()` ensures reproducibility of the sequence of random numbers. In general, you should always set the random number seed when conducting a simulation! Otherwise, you will not be able to reconstruct the exact numbers that you produced in an analysis.

2.1. Run the below codes two times, what is the result?

```
> set.seed(1)
> rnorm(5)
[1] -0.6264538  0.1836433 -0.8356286  1.5952808  0.3295078
> set.seed(1)
> rnorm(5)
[1] -0.6264538  0.1836433 -0.8356286  1.5952808  0.3295078
>
```

2.2. Now run the below codes two times, what are the results?

```
> rnorm(5)
[1] -0.8204684  0.4874291  0.7383247  0.5757814 -0.3053884
> rnorm(5)
[1]  1.5117812  0.3898432 -0.6212406 -2.2146999  1.1249309
```

2.3. Explain in your own words why the results in 2.1 and 2.2 are different.

The results in 2.1 and 2.2 are different because in 2.1, the code uses `set.seed(1)` before generating the random numbers, and this makes R start from the same point every time, so the same numbers are produced each time the code is run. In 2.2, there is no `set.seed()`, so R continues generating new random numbers each time. Because of that, the results change every time the code is run. In simple words, the seed makes the random results repeatable.

2.4. In real-world analytics, why is reproducibility important? Imagine you are running the analysis on your own and then run it again when presenting.

Reproducibility is important because it helps make the analysis consistent and trustworthy. If I run my analysis one day and then run it again later I want to get the same results. If the results change, it can make the analysis look unreliable or confusing.

In real-world analytics, reproducibility is important since it allows other people to verify the work, repeat the analysis, and trust the conclusions. It also helps avoid mistakes and makes the results easier to explain in business or professional settings.

3. The `sample()` function draws randomly from a specified set of (scalar) objects allowing you to sample from arbitrary distributions of numbers or letters.

3.1. Please try the following code and screenshot your result.

```
> set.seed(1)
> sample(1:10, 4)

> set.seed(1)
> sample(1:10, 4)
[1] 9 4 7 1
```

3.2. In the above `sample()` function, what does `1:10` mean?

`1:10` means the numbers from 1 through 10. This is the set of values that R is allowed to sample from.

3.3. In the above sample() function, what does 4 mean?

The 4 means that R should randomly choose 4 values from the set 1:10.

3.4. Now it is your turn to try to use sample () function to randomly draw 5 letters from the 26 letters. Please screenshot your code together with the result. (hints: letters/LETTERS)

```
> sample(letters, 5)
[1] "b" "w" "k" "n" "r"
>
```

3.5. If you run the same sample() code multiple times, will you always get the same result? Why or why not?

No, you won't always get the same result if you run the same sample() code multiple times. The result changes since the sampling is random. Each time R runs the code, it can choose different values. The only time you will always get the same result is if you use set.seed() before the sample() function.

3.6. How would you modify the sample() function to allow repeated letters? Explain what changes. You can use AI to help you to answer this question.

```
> sample(letters, 5, replace = TRUE)
[1] "s" "a" "u" "u" "j"
> |
```

Normally, sample() doesn't repeat values because if you replace = FALSE is the default. When we change it to replace = TRUE, every letter is put back after being selected, so the same letter can be chosen more than once (which means repeated letters are possible.)

Part II: Triangular Distribution

$$\text{Random observation} = \begin{cases} \min + \sqrt{(max - \min)(mode - \min)x} & \text{for } x < \frac{(mode - \min)}{(max - \min)} \\ max - \sqrt{(max - \min)(max - mode)(1 - x)} & \text{for } x \geq \frac{(mode - \min)}{(max - \min)} \end{cases}$$

In our in-class R activity, we used the numbers from the textbook example (handbag) to calculate the profit numbers for the three scenarios. In this part, we are using a triangular distribution to model profit because we do not have detailed historical data to estimate a full probability distribution. In many real-world situations (like this handbag manufacturer), we cannot precisely know the likelihood of every possible outcome. Instead, we rely on informed estimates: the worst-case scenario (minimum), the most likely outcome (mode), and the best-case scenario (maximum). The triangular distribution allows us to incorporate these three intuitive inputs into a simulation, giving us a reasonable approximation of uncertainty when data is limited. This is a common approach in business decision-making when historical data is unavailable or incomplete.

4.1. In the above code, explain in plain language what the ifelse() function is doing. Please interpret the meaning based on the variable in the function.

The ifelse() function is choosing which profit formula to use for each random number in the simulation. First, the code compares each value in randomNumbers with the threshold. If the random number is smaller than the threshold, R uses profitLTThreshold.

If the random number is bigger than or equal to the threshold, R uses profitGTThreshold. Which means the ifelse() function helps R decide which side of the triangular distribution the random number belongs to, and then it uses the correct formula to calculate the profit.

4.2. Based on your simulated mean profit, would you recommend proceeding with this project? Why or why not?

Yes, based on the simulated mean profit, I would recommend proceeding with the project, since the simulated mean profit is positive, that means that on average, the project is expected to make money. This shows that the project looks profitable overall and may be a good business opportunity. However, I would still be careful because there is still a chance of getting a low profit or even a loss in some cases, especially in the worst-case scenario.

4.3. Is the mean profit alone sufficient for decision-making? What additional information would you want?

No, the mean profit by itself is not enough for making a decision. The mean only shows the average result, but it doesn't show the risk of the project. A project can have a good average profit and still have some bad outcomes.

Some additional information I would need are the probability of loss, the minimum and maximum profit, the standard deviation, and maybe a graph or histogram of the profit results. This would help me better understand the risk before making a final decision.

4.4. Please provide screenshots of your codes and the results for this part.

```
> worst <- -4500
> best <- 119000
> mostLikely <- 44500
>
> threshold <- (mostLikely - worst) / (best - worst)
>
> set.seed(1)
> randomNumbers <- runif(100, 0, 1)
>
> profitLTThreshold <- worst + sqrt((best - worst) * (mostLikely - worst) * randomNumbers)
> profitGTThreshold <- best - sqrt((best - worst) * (best - mostLikely) * (1 - randomNumbers))
>
> profit <- ifelse(randomNumbers < threshold, profitLTThreshold, profitGTThreshold)
>
> mean(profit)
[1] 54184.75
> summary(profit)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  4502   39716   50353   54185   72719  110370
```